Estação Meteorológica Inteligente

Monitoramento ambiental para cidades inteligentes

Francisco Il Guk Fermone Castro de Souza — R330EC8
Lucas Vicente Pereira Silva — R4487J0
Matheus Bifaroni Carvalho — R954838

UNIP · Ciência da Computação, 2º semestre
Projeto e Desenvolvimento II (68F3)
Professor Marco Gomes · São Paulo, 2026

Vídeo técnico: assistir à demonstração completa

Monitoramento ambiental local

O problema

As condições ambientais variam entre bairros. Informações locais acessíveis ajudam a população a acompanhar o microclima.

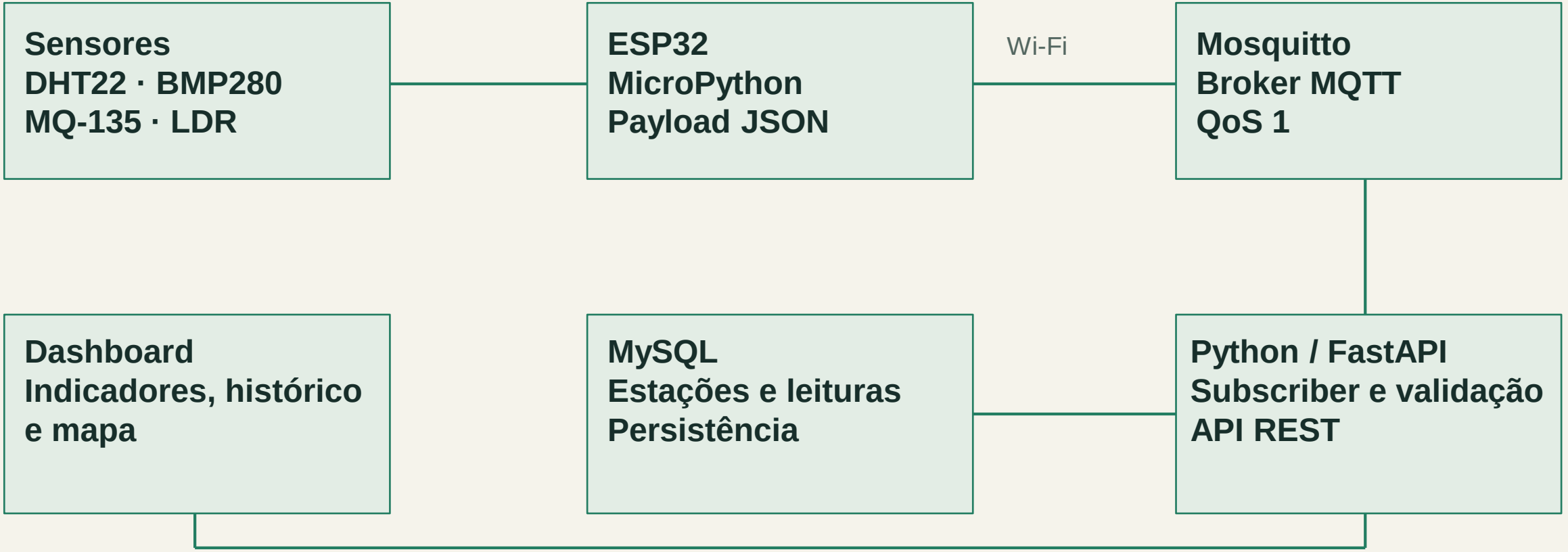
A contribuição

Uma cidade inteligente usa dados para apoiar decisões urbanas. A estação reúne leituras, histórico e mapa para acompanhar o microclima, ampliar a conscientização ambiental e discutir ações sustentáveis.

Base de IoT: Oliveira (2017). Aplicação e registros: equipe.

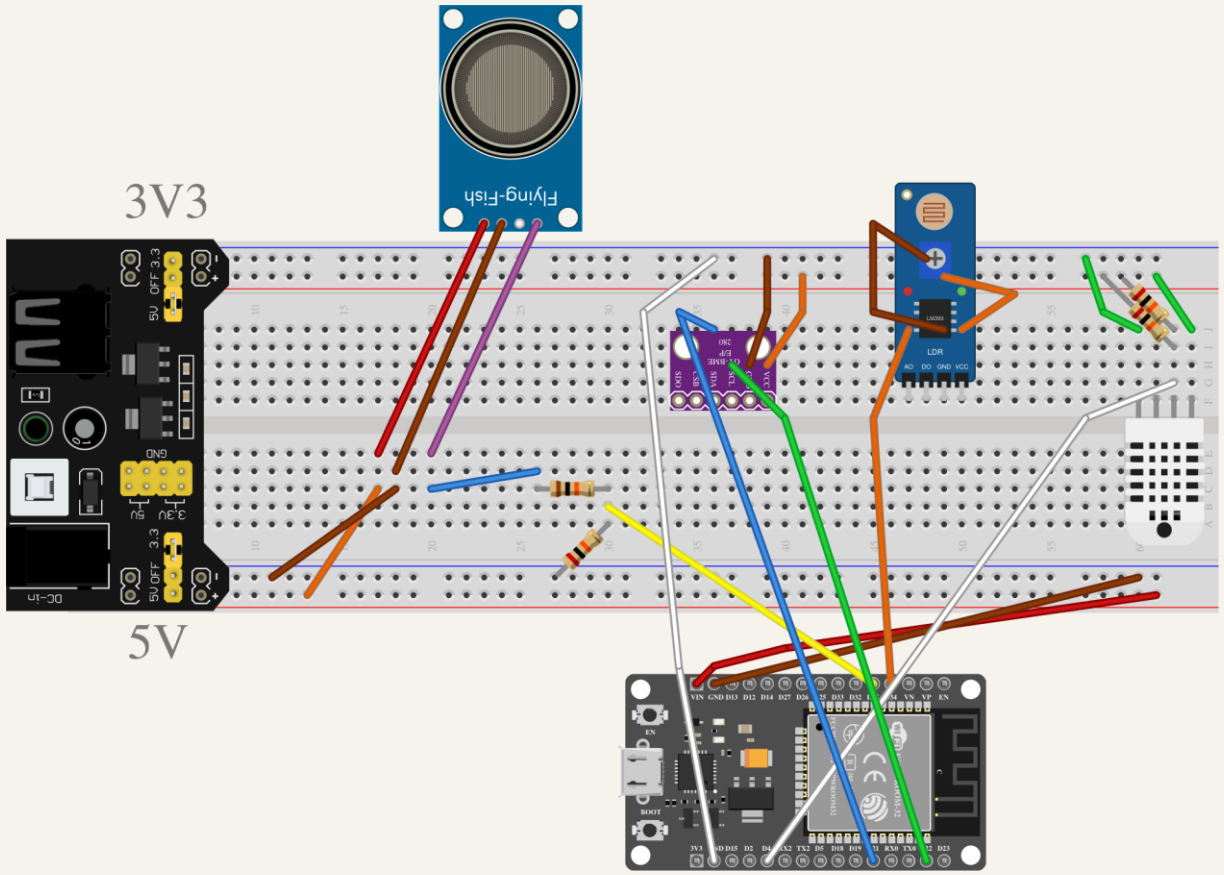
Arquitetura da solução

Base: Oliveira (2017) e Kolban (2018). Implementação: firmware e backend da equipe.



O dashboard consulta a API por HTTP/JSON; o backend acessa o banco.

Hardware e montagem no Fritzing



DHT22: DATA em GPIO4
BMP280: SDA21 / SCL22
LDR: AO em GPIO34
MQ-135: AO em GPIO35

HW-131: 5 V para VIN e MQ-135; 3,3 V para os demais módulos.
GND comum com a ESP32.

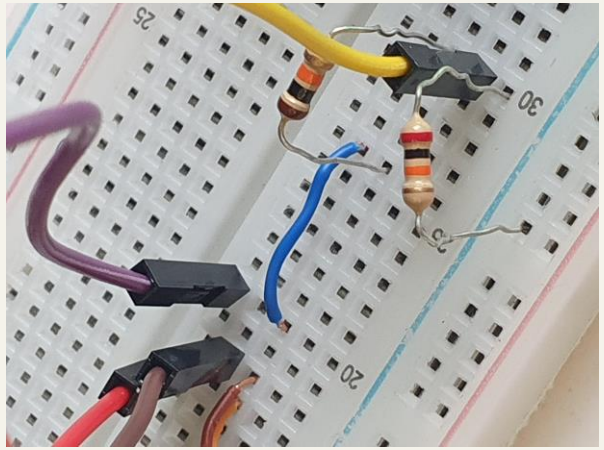
DHT22: pull-up de 10 kΩ, formado por dois resistores de 20 kΩ em paralelo.

MQ-135: divisor de 10 kΩ / 20 kΩ; D35 no ponto central entre os resistores.

Base: Idoeta e Capuano (2019); Lourenço et al. (2009). Esquema: equipe. Pluviômetro dispensado conforme correções.

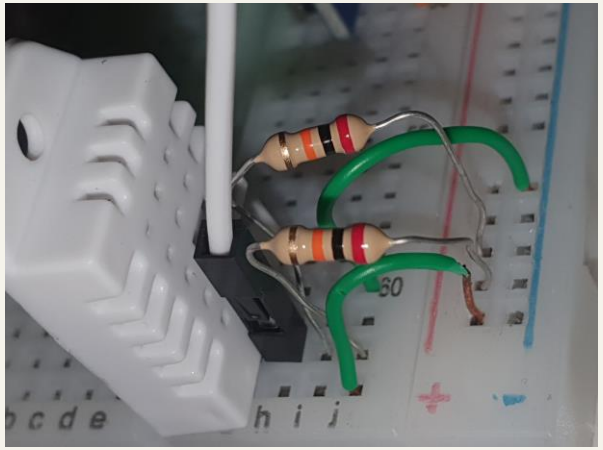
Condicionamento dos sensores

MQ-135 · divisor de tensão



AO — 10 kΩ — D35 — 20 kΩ — GND
A entrada D35 recebe 2/3 da tensão AO.
Fator de reconstrução: 1,5.
O divisor reduz a tensão no ADC.

DHT22 · pull-up do DATA



20 kΩ ‖ 20 kΩ = 10 kΩ
Ligação entre DATA / GPIO4 e 3,3 V.
Mantém nível alto quando a linha
não está sendo acionada.

ADC: 12 bits, 0 dB e read_uv(). PPM estimado com referência virtual de 0,045 V.
Fotos: equipe. Fontes técnicas: MicroPython, Espressif, Winsen e MQSensorsLib.

Firmware MicroPython e telemetria

Coleta a cada 2,5 segundos

Tarefas assíncronas separam aquisição, Wi-Fi e publicação MQTT.
O payload JSON inclui leituras, ADC, horário, boot_id e sequência.

Fila em RAM com até 24 amostras para interrupções curtas.
Reconexão sem interromper o ciclo principal de aquisição.

WebREPL permite acompanhar os logs pelo Wi-Fi.
As leituras não dependem de o navegador permanecer aberto.

Base de algoritmos: Menezes (2019); Ascencio e Campos (2012). ESP32: Kolban (2018).

Backend: recebimento, validação e persistência

Base: Alencar Filho (2008), Menezes (2019) e Ascencio e Campos (2012). Código: equipe.

```python
def on_message(client, userdata, message):
    try:
        if message.retain:
            log.warning("Mensagem retida ignorada: %s", message.topic)
            client.ack(message.mid, message.qos)
            return
        if len(message.payload) > 8192:
            raise ValueError("Payload excede 8192 bytes")
        dados = TelemetriaSchema.model_validate_json(message.payload)
        if message.topic != f"estacoes/{dados.id_estacao}/telemetria":
            raise ValueError("Topico diverge da estacao")
        log.info("MQTT recebido estacao=%s seq=%s", dados.id_estacao, dados.sequencia)
        persistir(dados)
        client.ack(message.mid, message.qos)
    except (ValidationError, ValueError) as exc:
        log.warning("Telemetria rejeitada: %s", exc)
        client.ack(message.mid, message.qos)
    except Exception:
        log.exception("Persistencia MQTT falhou; mensagem sem confirmacao")
```

```
2026-11-08 16:00:28,629 INFO [ESTACAO] Telemetria persistida estacao=1 seq=312 duracao_ms=13.0 filtros={}
2026-11-08 16:00:28,632 INFO [ESTACAO] MQTT recebido estacao=1 seq=313
2026-11-08 16:00:28,633 INFO [ESTACAO] SQL consulta=aeeb6df3 duracao_ms=0.62
2026-11-08 16:00:28,634 INFO [ESTACAO] SQL consulta=69c842de duracao_ms=0.53
2026-11-08 16:00:28,635 INFO [ESTACAO] SQL consulta=1fa8a823 duracao_ms=0.65
2026-11-08 16:00:28,636 INFO [ESTACAO] SQL consulta=d56a0e8e duracao_ms=0.86
2026-11-08 16:00:28,637 INFO [ESTACAO] SQL consulta=5cd8f462 duracao_ms=0.80
2026-11-08 16:00:28,643 INFO [ESTACAO] Telemetria persistida estacao=1 seq=313 duracao_ms=11.0 filtros={}
2026-11-08 16:00:28,646 WARNING [ESTACAO] Telemetria rejeitada: 1 validation error for TelemetriaSchema
temperatura
  Input should be less than or equal to 80 [type=less_than_equal, input_value=150, input_type=int]
    For further information visit https://errors.pydantic.dev/2.14/v/less_than_equal
2026-11-08 16:00:28,647 INFO [ESTACAO] MQTT recebido estacao=1 seq=314
2026-11-08 16:00:28,648 INFO [ESTACAO] SQL consulta=aeeb6df3 duracao_ms=0.75
2026-11-08 16:00:28,649 INFO [ESTACAO] SQL consulta=69c842de duracao_ms=0.54
2026-11-08 16:00:28,650 INFO [ESTACAO] SQL consulta=1fa8a823 duracao_ms=0.65
2026-11-08 16:00:28,652 INFO [ESTACAO] SQL consulta=d56a0e8e duracao_ms=0.87
2026-11-08 16:00:28,653 INFO [ESTACAO] SQL consulta=5cd8f462 duracao_ms=0.80
2026-11-08 16:00:28,659 INFO [ESTACAO] Telemetria persistida estacao=1 seq=314 duracao_ms=12.0 filtros={}
2026-11-08 16:00:28,661 INFO [ESTACAO] MQTT recebido estacao=1 seq=315
```

Validação: temperatura finita e $-40 \leq T \leq 80$ °C.
Mediana e desvio absoluto mediano detectam saltos discrepantes.

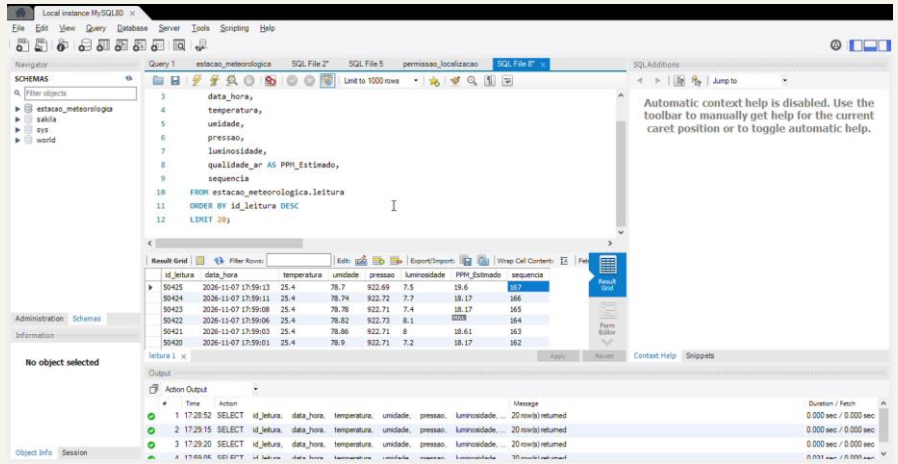
Média de até cinco amostras suaviza temperatura, umidade e pressão.
Payload original e flags de qualidade ficam registrados no MySQL.

Modelagem de dados: estação e leituras

ESTAÇÃO 1 N **LEITURA**

PK id_estacao: INT
latitude: FLOAT NOT NULL
longitude: FLOAT NOT NULL
descricao: VARCHAR(100)

PK id_leitura: BIGINT
FK id_estacao: INT NOT NULL
data_hora: TIMESTAMP
sensores: FLOAT NULL
boot_id: VARCHAR(32)
sequencia: BIGINT
payload / qualidade: JSON



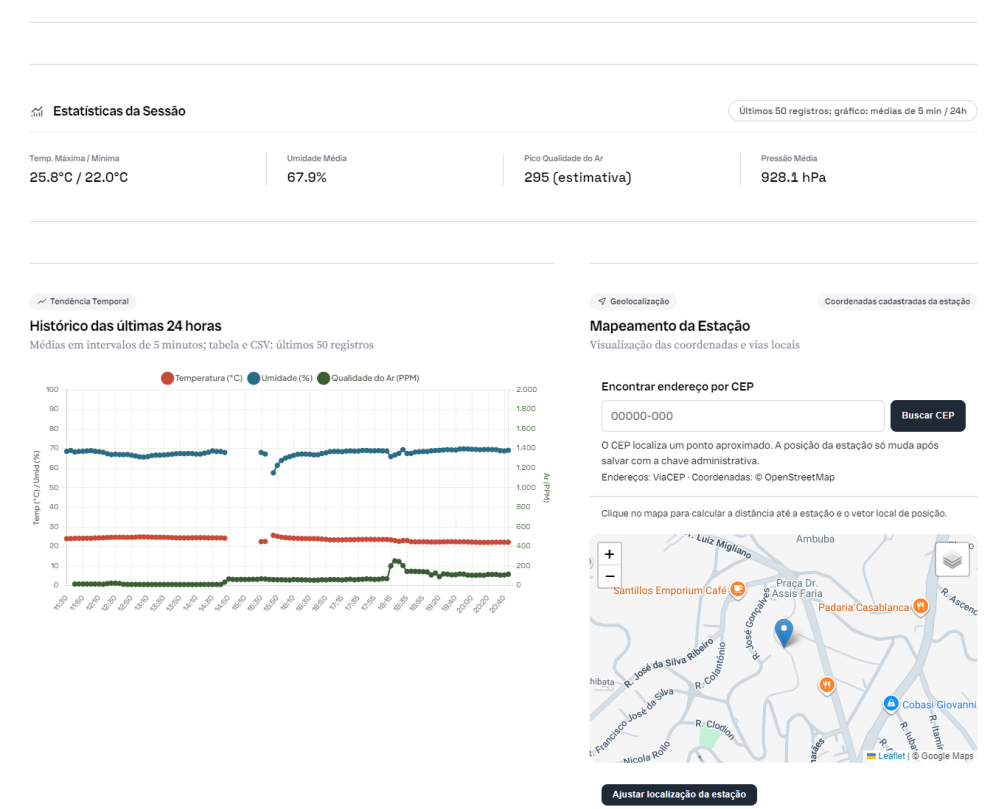
Consulta SQL e registros reais.
Captura da gravação do MySQL.

Uma estação possui várias leituras. Unicidade: estação + boot + sequência.

PK, FK e UNIQUE instalados. CHECK previstos no DDL; validação de limites no backend.

Fontes: database/estacao_meteorologica.sql e vídeo MySQL da equipe, 7 nov. 2026.

Dashboard e geometria aplicada ao mapa



Indicadores com unidade e horário.
Histórico de leituras e alertas.

Marcador em latitude/longitude.
Vetor local: v = (ΔE, ΔN).
Distância geográfica: Haversine.

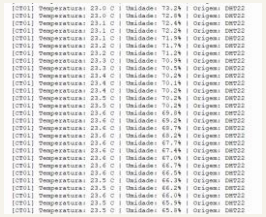
Demonstrações são identificadas.
Avisos reais acompanham a telemetria recebida.

Base: Winterle (2014). Captura: equipe. Localização manual, sem GPS embarcado.

Resultados dos testes CT01–CT04

CT01 · DHT22

Aprovado: 22,5 → 23,6 °C (+1,1 °C), com redução após o estímulo.



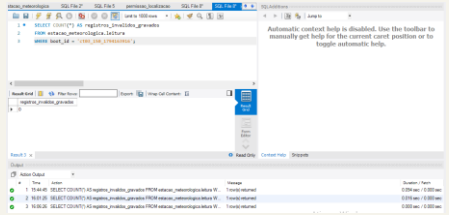
CT02 · Comunicação

Aprovado: ausência de dados recentes durante a interrupção e retomada após reconexão.



CT03 · Validação

Aprovado: T = 150 °C rejeitada no backend; consulta do teste retornou zero registros inválidos.



CT04 · Localização

Aprovado: marcador exibido na região cadastrada para os testes.



Fonte: evidências da equipe, 8 nov. 2026. Capturas e vídeos integrais acompanham o relatório.

Conclusão, desafios e próximos passos

Resultado alcançado

O protótipo integra aquisição, envio MQTT, validação, armazenamento e visualização. Os testes registraram resposta térmica, reconexão, rejeição de dado inválido e localização no mapa.

Desafios e soluções aplicadas

Linha digital: pull-up no DHT22
Entrada analógica: divisor no MQ-135
Dados discrepantes: filtros no backend
Perda de rede: fila e reconexão

Continuidade do projeto

Caixa hermética IP65 impressa em 3D
Painel solar e bateria Li-Ion
Rede mesh com múltiplas estações
Machine Learning: previsão do tempo e alertas preditivos de alagamento

Limites: PPM/lux estimados, contatos da protoboard e dependência dos serviços no notebook.

Integração de Python, Circuitos Digitais, Lógica Matemática, Geometria Analítica e Banco de Dados.

Referências bibliográficas

ALENCAR FILHO, Edgard de. **Iniciação à lógica matemática**. São Paulo: Nobel, 2008.

ASCENCIO, A. F. G.; CAMPOS, E. A. V. de. **Fundamentos da programação de computadores**: algoritmos, Pascal, C/C++ e Java. 3. ed. São Paulo: Pearson, 2012.

CALIFA, Miguel A. et al. **MQ-135-ALL**: MQSensorsLib. GitHub, 2020. Disponível em: https://github.com/miguel5612/MQSensorsLib. Acesso em: 8 nov. 2026.

ESPRESSIF SYSTEMS. **Analog to Digital Converter (ADC)**: ESP32. Disponível em: https://docs.espressif.com/projects/esp-idf/en/stable/esp32/api-reference/peripherals/adc.html. Acesso em: 8 nov. 2026.

IDOETA, I. V.; CAPUANO, F. G. **Elementos de eletrônica digital**. 42. ed. São Paulo: Érica, 2019.

KOLBAN, Neil. **Kolban's book on ESP32**. [S. l.]: Leanpub, 2018. Disponível em: https://leanpub.com/kolban-ESP32. Acesso em: 8 nov. 2026.

LOURENÇO, A. C. de et al. **Circuitos digitais**: estude e use. 9. ed. São Paulo: Érica, 2009.

MENEZES, N. N. C. **Introdução à programação com Python**: algoritmos e lógica de programação para iniciantes. 3. ed. São Paulo: Novatec, 2019.

MICROPYTHON. **Quick reference for the ESP32**. Disponível em: https://docs.micropython.org/en/latest/esp32/quickref.html. Acesso em: 8 nov. 2026.

OLIVEIRA, Sérgio de. **Internet das Coisas com ESP8266, Arduino e Raspberry Pi**. São Paulo: Novatec, 2017.

WINTERLE, Paulo. **Vetores e geometria analítica**. 2. ed. São Paulo: Pearson, 2014.

ZHENGZHOU WINSEN ELECTRONICS TECHNOLOGY. **MQ135 semiconductor sensor for air quality**: manual. Disponível em: https://www.winsen-sensor.com/d/files/manual/mq135.pdf. Acesso em: 8 nov. 2026.

Bibliografia curricular e fontes técnicas complementares. Ordem alfabética por coluna.